



MASTERING THE K-NEAREST NEIGHBORS ALGORITHM

Houmir Yassine 1820249035

Pion Mathieu 1820249034

Nguyen Etienne 1820249033



INTRODUCTION TO KNN

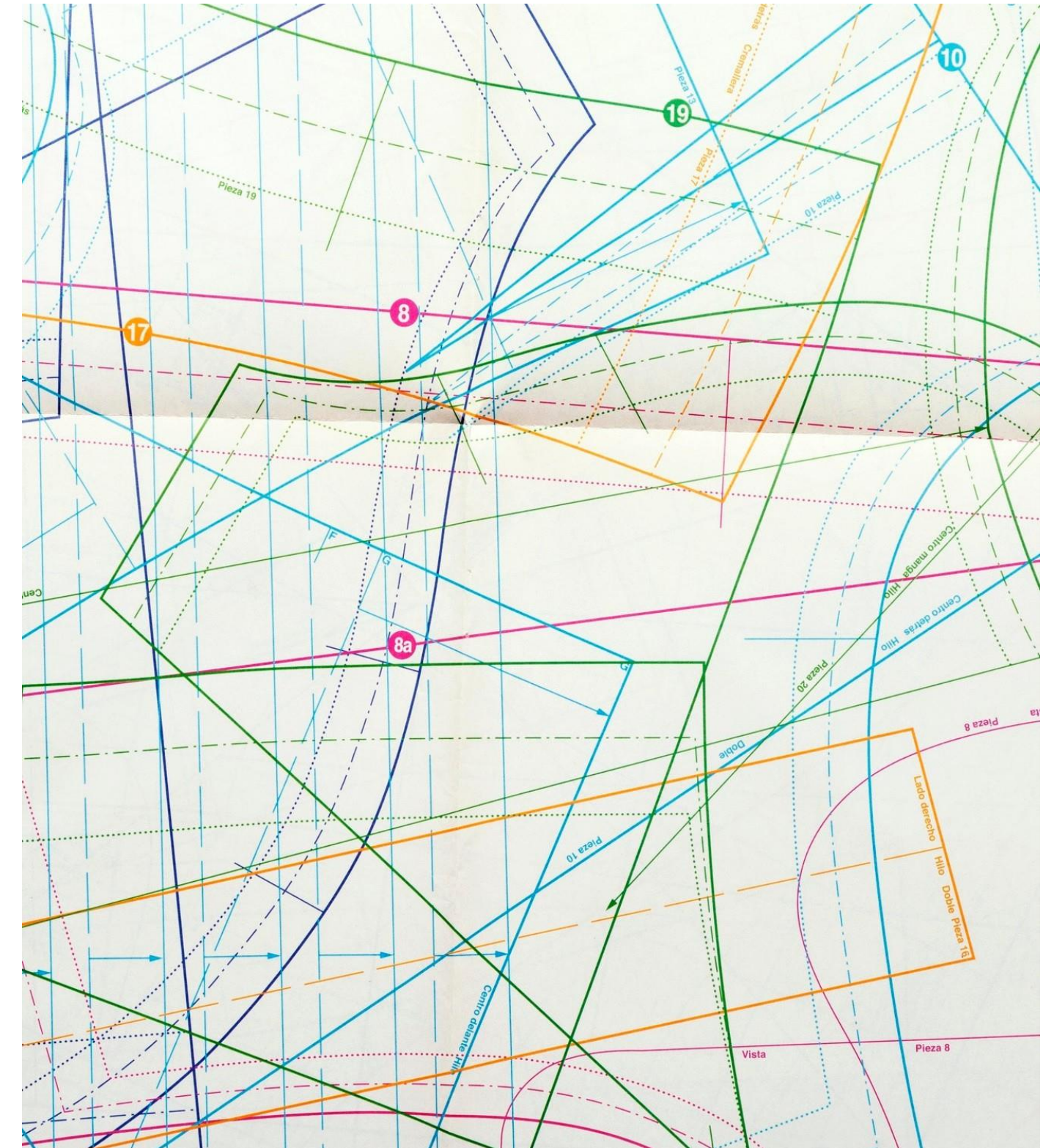
K-Nearest neighbors (KNN) is a simple yet powerful algorithm used for **Classification** and **regression** tasks. This presentation will delve into its code implementation and practical examples, showcasing how KNN can be effectively utilized in real-world scenarios.

Why Choose KNN?

K-Nearest Neighbors (KNN) is a non-parametric and instance-based learning algorithm, which makes it versatile and simple to use. Unlike other algorithms, KNN doesn't make assumptions about the data distribution, making it suitable for a variety of tasks. It's intuitive and easy to understand, as it bases predictions on the proximity of data points, often leading to high interpretability. However, KNN can be computationally expensive on large datasets, and its performance may decrease with noise in the data.

Basic Principles of KNN

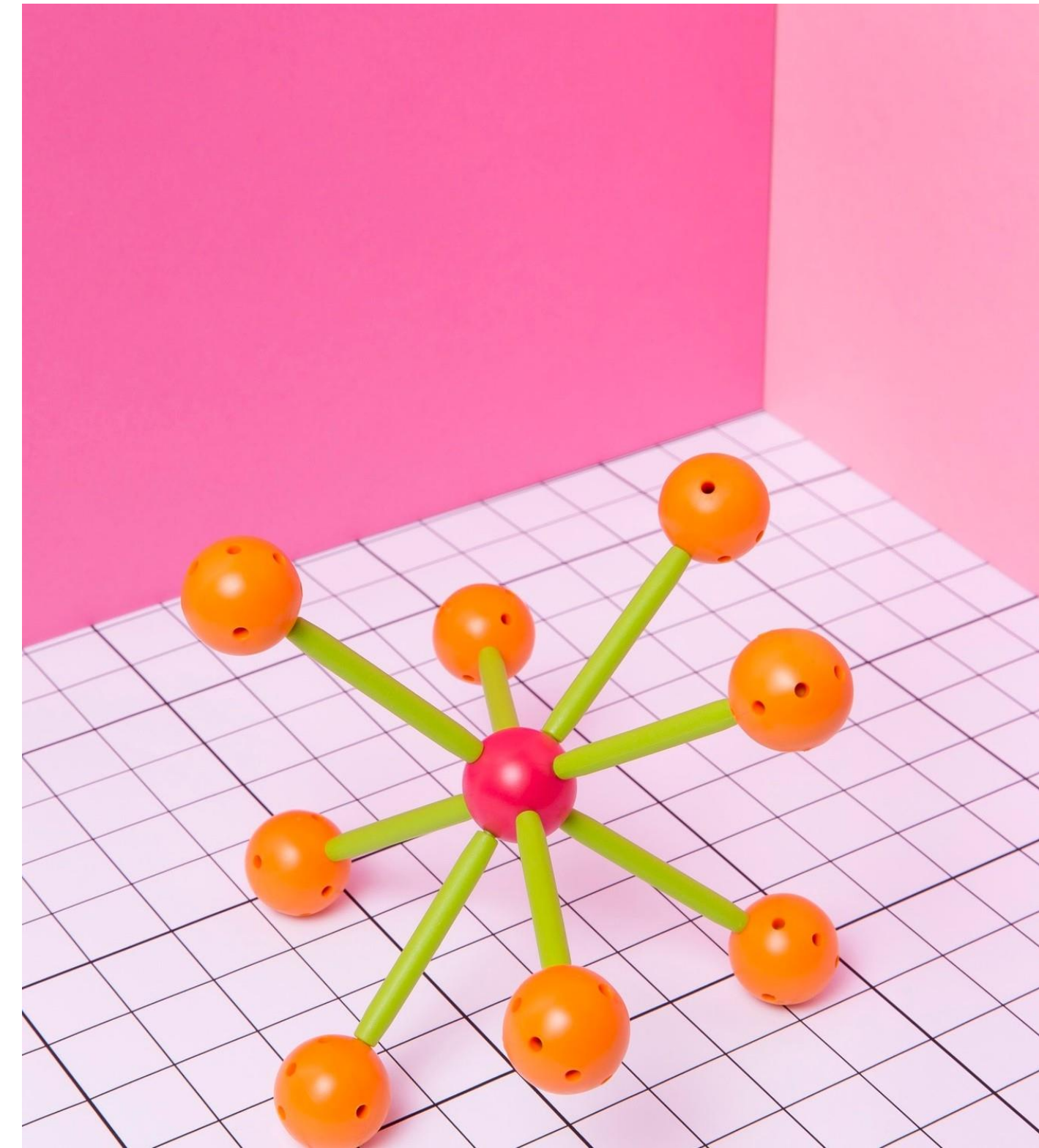
The **K-Nearest Neighbors** algorithm classifies data points based on their **proximity** to other points. It operates on **feature** the principle that similar data points are located near each other in the **space**, making it intuitive and easy to understand.



HOW KNN WORKS

KNN works by calculating the **distance** between a query point and all other points in the dataset.

The algorithm then selects the **K-nearest neighbors** and assigns the most common class among them to the query point, making it a **non-parametric** method.



DISTANCE METRICS

KNN can utilize various **distance metrics** to determine proximity, including :

- Euclidean**
- Manhattan**
- Minkowski**

The choice of metric can significantly affect the algorithm's performance and should align with the dataset's characteristics.

Distance functions

Euclidean

$$\sqrt{\sum_{i=1}^k (x_i - y_i)^2}$$

Manhattan

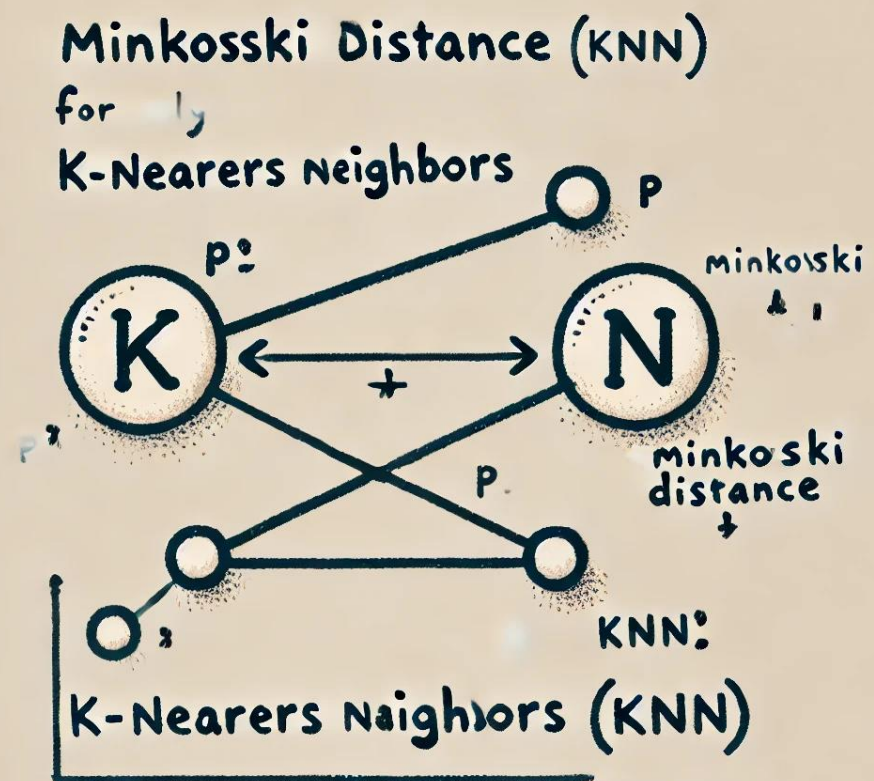
$$\sum_{i=1}^k |x_i - y_i|$$

Minkowski

$$\left(\sum_{i=1}^k (|x_i - y_i|)^q \right)^{1/q}$$

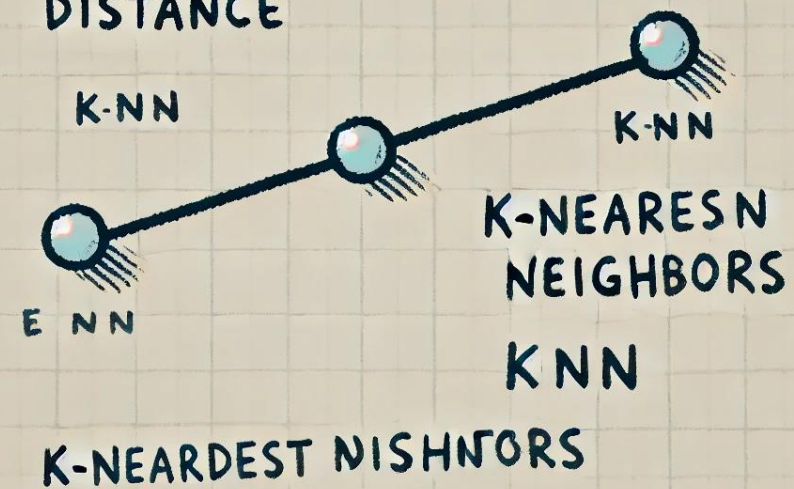
Choosing the Path: Exploring KNN Distance Metrics

Minkowski

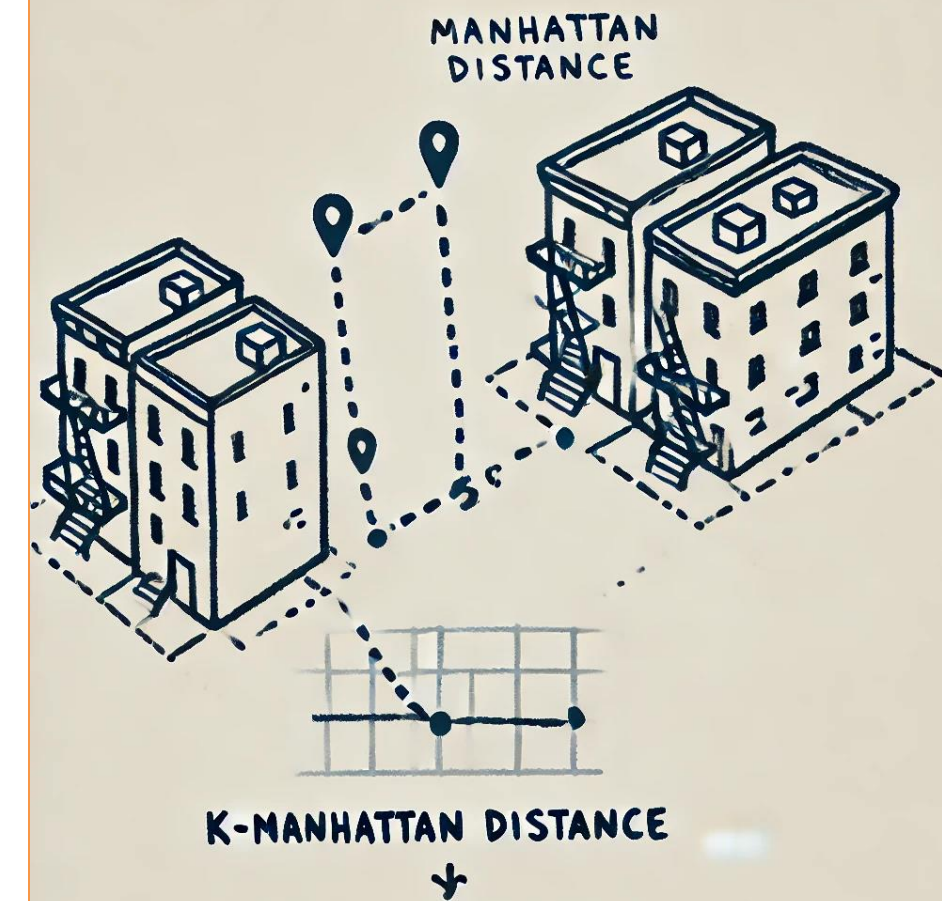


Euclidean

EUCLIDEAN
DISTANCE



Manhattan





CHOOSING THE RIGHT K

Selecting the appropriate value of K is crucial for KNN's performance.

Small K : may lead to noise sensitivity

Large K : may smooth out important patterns.

Techniques such as **cross-validation** can help determine the best K for a specific dataset.

DATA PREPROCESSING

Before implementing KNN, it's essential to perform **data preprocessing**.

This includes :

- handling missing values
- normalizing features
- encoding categorical variables.

Proper preprocessing ensures that the algorithm works effectively and yields accurate results.



CODE IMPLEMENTATION: SETUP

To implement KNN in Python, we typically use libraries like **scikit-learn**

Begin by importing necessary packages, loading the dataset, and splitting it into **training** and **testing** sets.

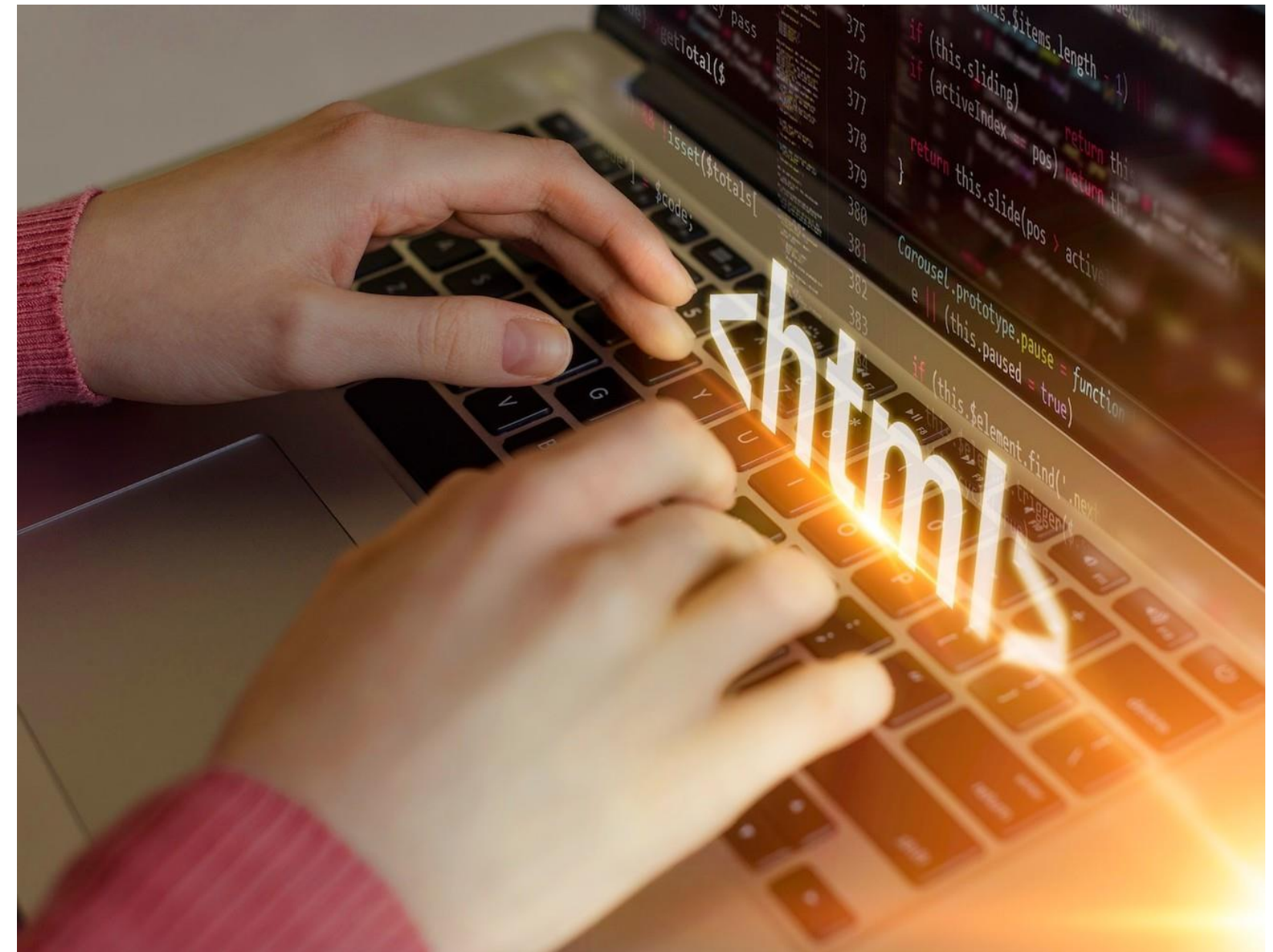
This setup forms the foundation for applying the KNN algorithm.



CODE IMPLEMENTATION: TRAINING

After setting up the data, we can train the KNN model using the **fit** method.

The model learns from the training data, establishing the relationships between the features and their corresponding labels, preparing for predictions.



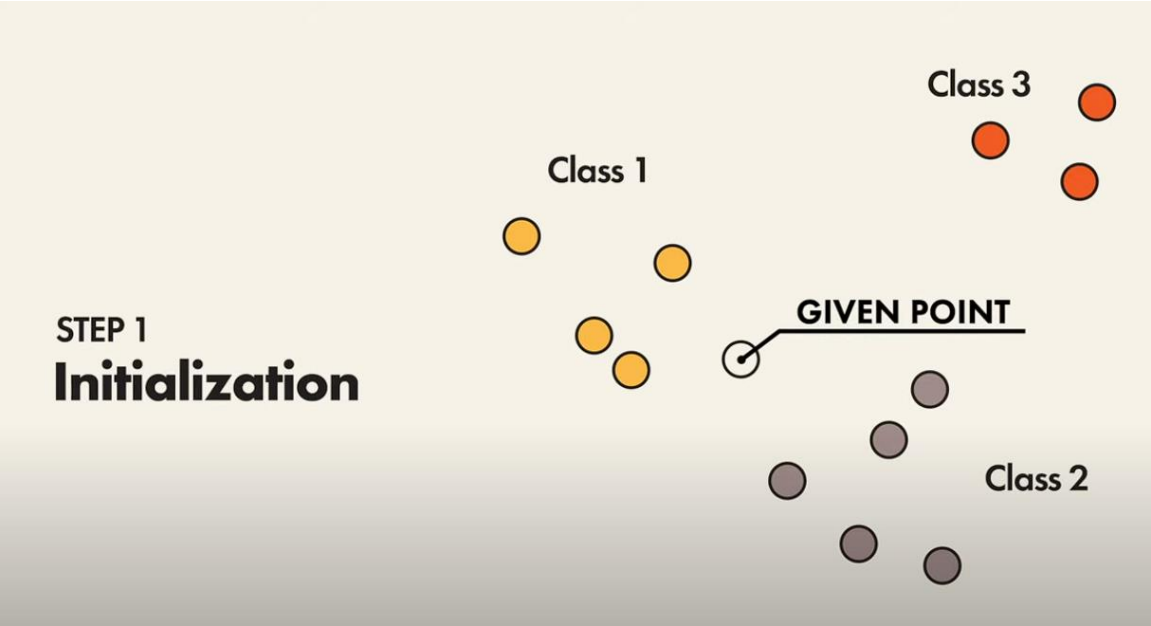
CODE IMPLEMENTATION: PREDICTION

Once the model is trained, we can use the **predict** method to classify new data points.

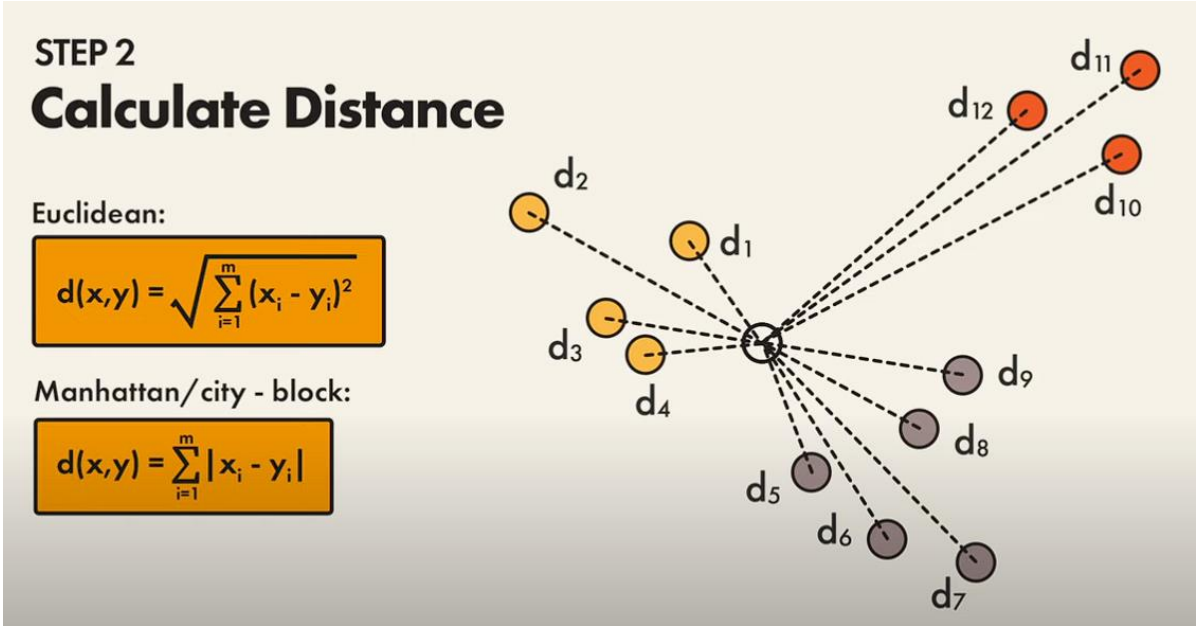
The trained KNN model identifies the nearest neighbors and assigns the class based on the majority vote among them, providing a prediction.



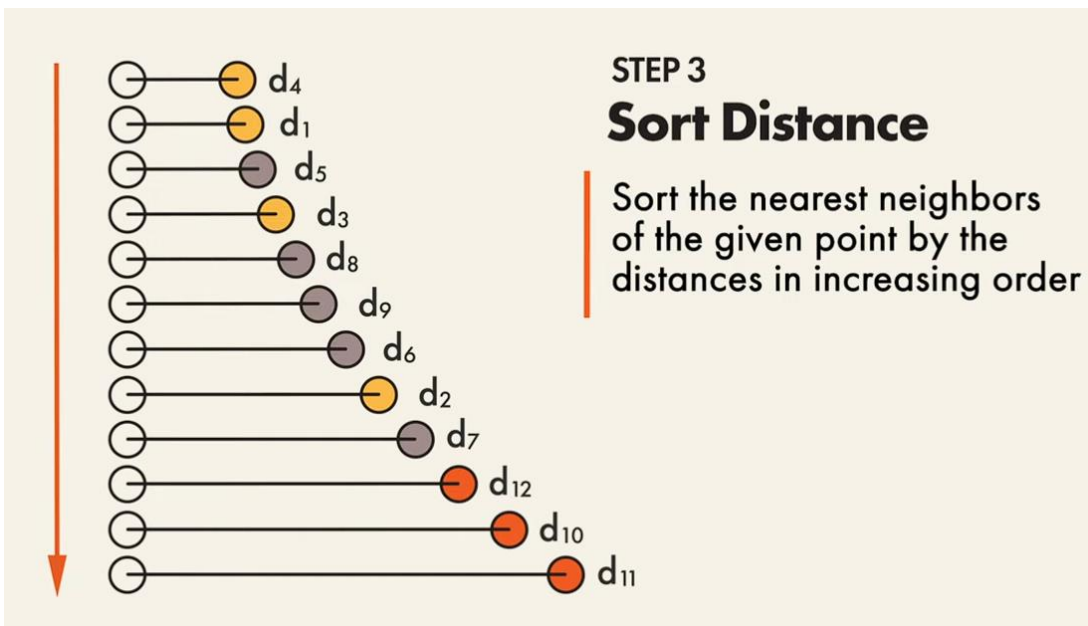
Practical Example:



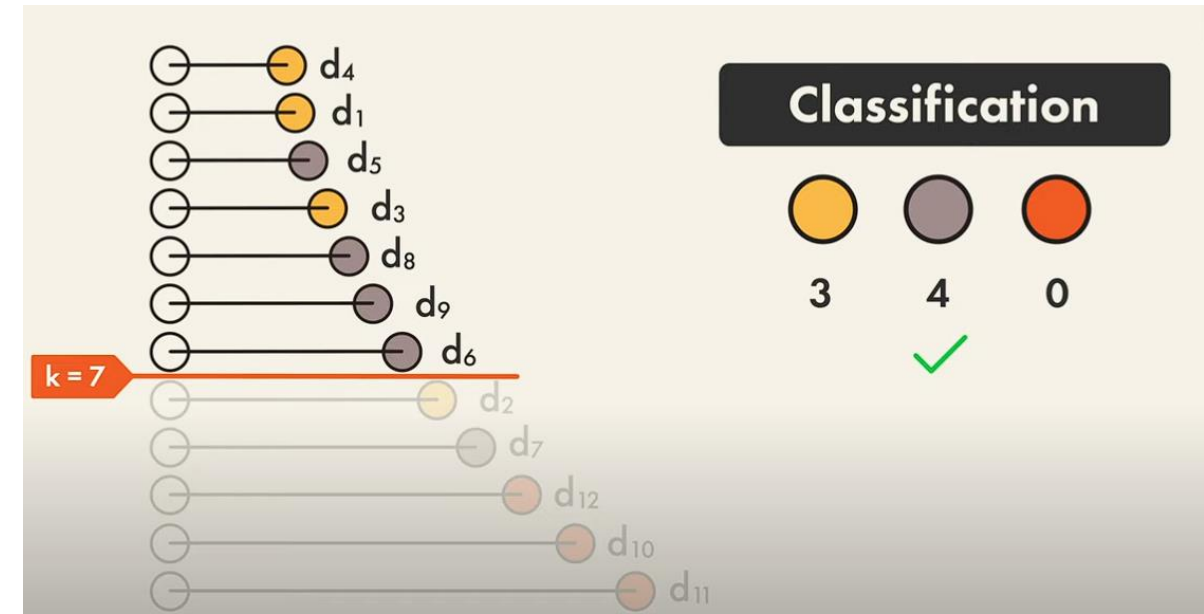
Here it's a simple two-dimensional example for you to have a better understanding of this algorithm. Let's say we want to classify the given point into one of the three groups.



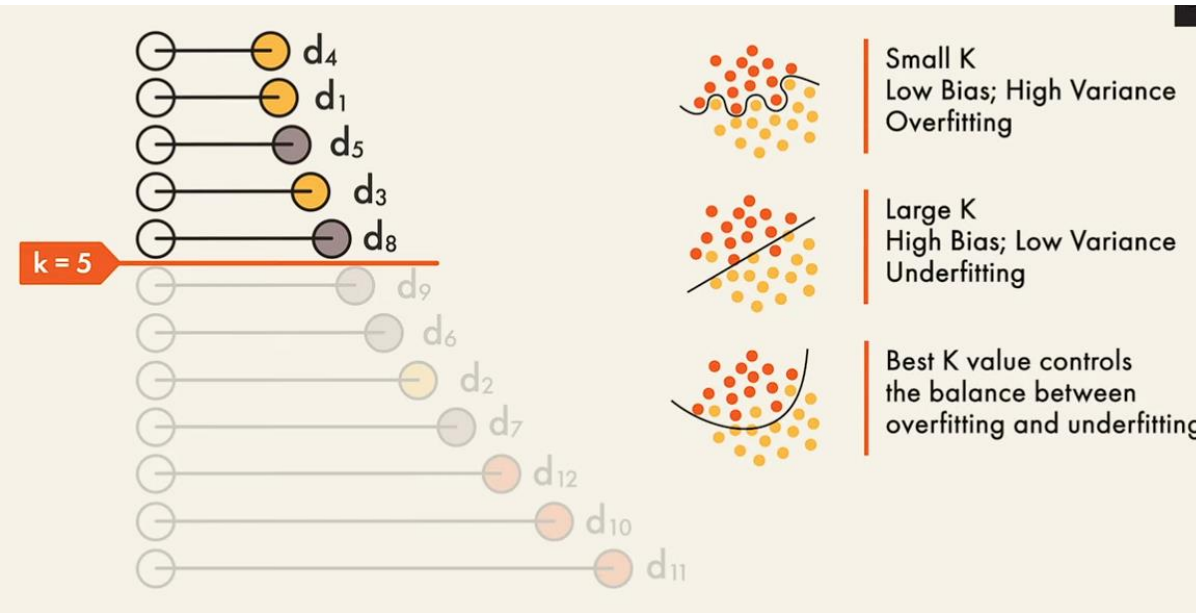
To find the k nearest neighbors of the given point, we need to calculate the distance between the given point to the other points. There are many distance functions but Euclidean is the most used one.



Then, we need to sort the nearest neighbors of the given point by the distances in increasing order.



For the classification problem, the point is classified by a vote of its neighbors, then the point is assigned to the class most common among its k nearest neighbors.

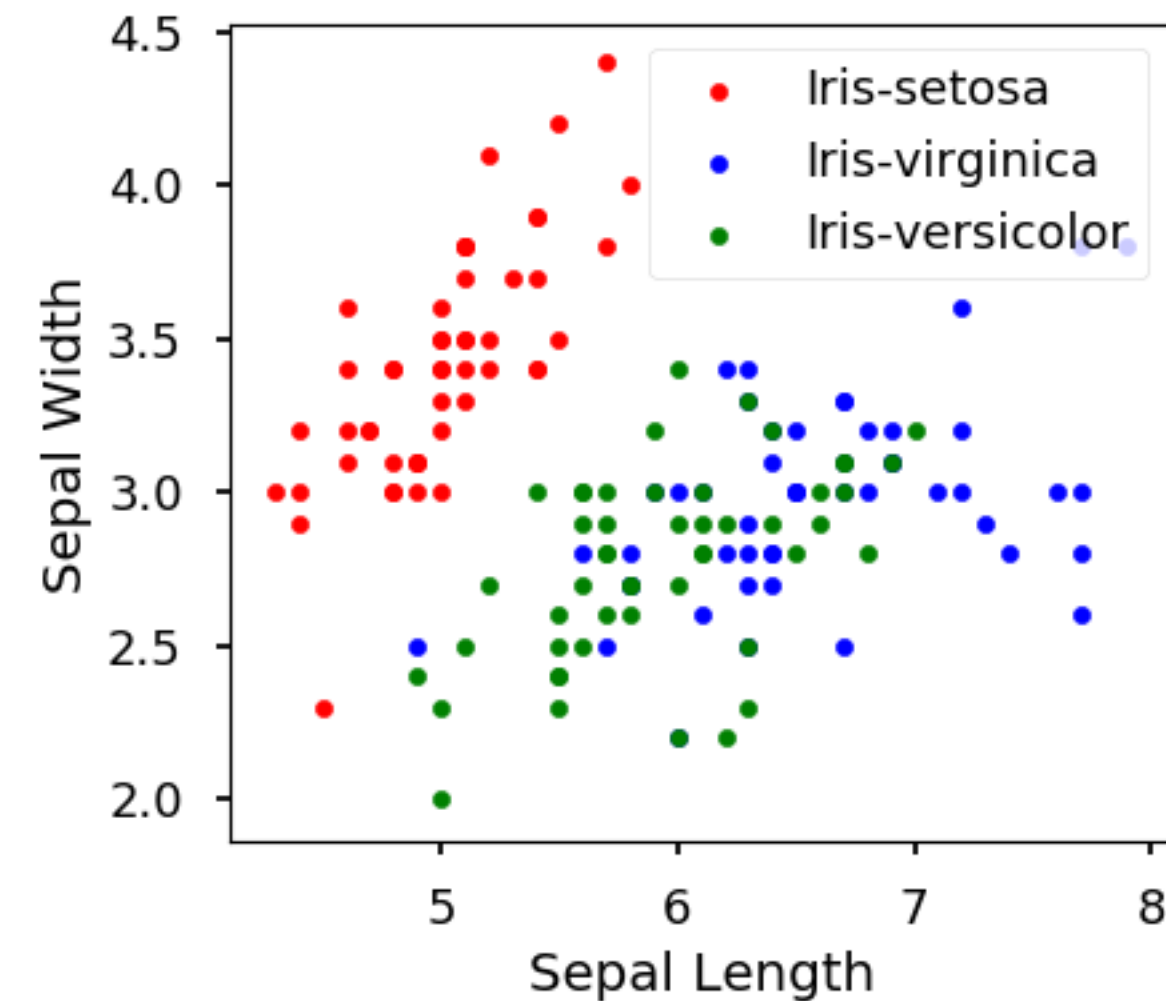
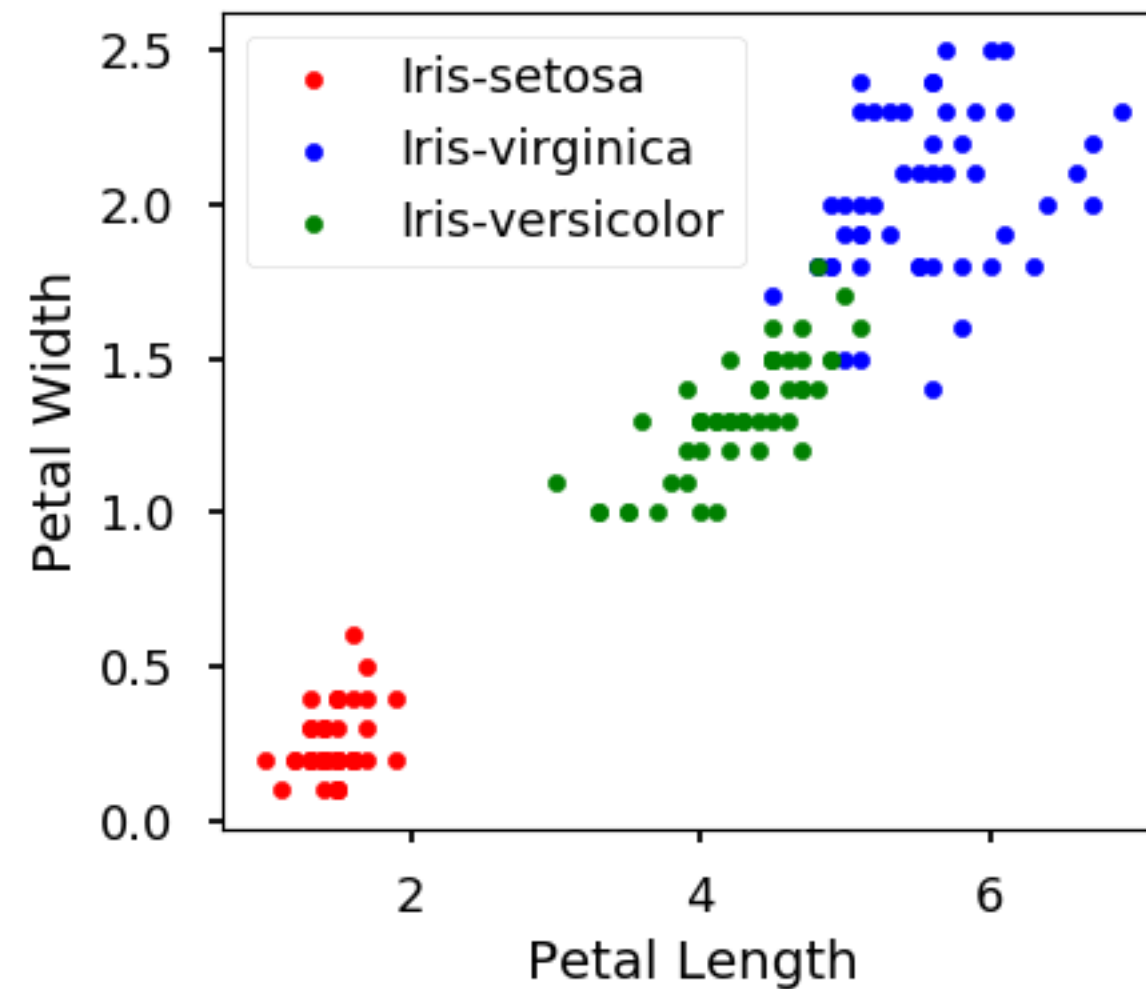


K value here control the balance between overfitting and underfitting, the best value can be found with cross validation and learning curve. A small k value usually leads to low bias but high variance, and a large k usually leads to high bias but low variance, it is important to find a balance between them.

PRACTICAL EXAMPLE: CLASSIFICATION

KNN can be utilized for classification task such as identifying species of **flowers based** on features like petal and sepal dimensions.

This example demonstrates KNN's effectiveness in solving real-world problems.



PRACTICAL EXAMPLE: REGRESSION

KNN can also be applied to regression problems, such as predicting house prices based on various features like size and location. We simply return the average of the k nearest neighbors' labels as the prediction.

This showcases KNN's versatility and ability to adapt to different types of predictive modeling tasks.

Regression

$$\hat{y} = \frac{y_4 + y_1 + y_5 + y_3 + y_8}{k}$$

Classification/Regression

Classification	Regression
Flower Species Identification	<i>House Price Prediction</i>
<i>Spam Detection</i>	<i>Energy Consumption Estimation</i>
<i>Image Categorization</i>	<i>Stock Price Prediction</i>

KNN Versatility: The K-Nearest Neighbors algorithm is highly versatile, making it suitable for both classification and regression tasks.

Classification: KNN assigns discrete labels by finding the closest matching examples. For instance, it can identify if an email is spam or classify flower species based on features like petal and sepal lengths.

Regression: KNN can predict continuous values by averaging the outcomes of the nearest neighbors. Examples include estimating house prices or forecasting energy usage.

Data Preprocessing

```
from sklearn.preprocessing import StandardScaler, OneHotEncoder
from sklearn.impute import SimpleImputer

scaler = StandardScaler() # Scale features
X_scaled = scaler.fit_transform(X)

imputer = SimpleImputer(strategy='mean') #Handle missing values
X_imputed = imputer.fit_transform(X)

encoder = OneHotEncoder() # Encode categorical data
X_encoded = encoder.fit_transform(categorical_features)
```

Preprocessing steps for KNN:

Handle Missing Values:

- Replace missing data (e.g., mean/median) or remove incomplete rows.

Feature Scaling:

- Ensure all features are on the same scale (Standardization/Normalization).

Encoding Categorical Variables:

- Convert non-numeric data to numeric using methods like One-Hot Encoding.

Best Practices:

- **Remove Outliers:** Prevent skewed distance calculations.
- **Balanced Classes:** Avoid biased predictions in classification.

Python Code Explanation

```
# Import necessary libraries
from sklearn.neighbors import KNeighborsClassifier
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler

# Load and preprocess the data
# Step 1: Data preprocessing - scaling features
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

# Step 2: Split the dataset into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(X_scaled, y, test_size=0.3, random_state=42)

# Step 3: Train the KNN model with k=3
knn = KNeighborsClassifier(n_neighbors=3)
knn.fit(X_train, y_train)

# Step 4: Make predictions on new data
predictions = knn.predict(X_test)

accuracy = knn.score(X_test, y_test) # Evaluate the model's performance
print(f'Accuracy: {accuracy * 100:.2f}%')
```

Step 1: Import Libraries :

-Necessary tools for implementing KNN (ex: KNeighborsClassifier, train_test_split)

Step 2: Preprocessing :

-Scale features to ensure consistent distance measurement.

Step 3: Train the Model :

-Use the training set to fit the KNN model, setting k=3.

Step 4: Make Predictions :

-Predict classes for new data based on the closest 3 neighbors.

Evaluate Performance :

-Check the model's accuracy using test data.

Key Takeaway: This step-by-step process helps build a reliable KNN classifier by following best practices in data preparation and model evaluation.

Real-World Applications



KNN's simplicity and versatility make it useful in numerous fields. In finance, KNN is applied for fraud detection by comparing new transactions with past patterns. In healthcare, it's used to predict disease likelihood based on patient history and similar cases. In image recognition, KNN helps classify objects by matching features with known images. These applications showcase how KNN can be adapted to solve a wide range of problems by finding similarities in data



Advantages

Simplicity: Easy to implement and understand, making it suitable for quick tasks and small datasets.



Non-parametric: No assumptions about data distribution, which can improve flexibility.

Versatility: Works for both classification and regression tasks.

Disadvantages

Computationally Intensive: Requires storing and processing the entire dataset, making it slow for large datasets.

Sensitive to Irrelevant Features: Performance can be affected by noisy or irrelevant features, requiring careful preprocessing.

Poor Performance in High Dimensions: Struggles with high-dimensional data due to the "curse of dimensionality."

CONCLUSION

In conclusion, mastering the K-Nearest Neighbors algorithm equips you with a powerful tool for both classification and regression tasks.

By understanding its mechanics, implementation, and practical applications, you can leverage KNN to tackle various data challenges effectively.